

PROJECT REPORT

COURSE PREREQUISITE PLANNER

Web-Based Graph Algorithm Application

Prepared by: A. Vaishali

A.keerthi

B.Manasa

K.Spandana

GitHub: anamagandlavaishali-gif

Repository: Course-Prerequisite-Planner

Technology: HTML, CSS, JavaScript, SVG

1. Abstract

The Course Prerequisite Planner is an interactive web application designed to help students understand and plan university courses based on prerequisite relationships. Courses are represented as nodes in a directed graph and prerequisite relationships are represented as directed edges. The application uses graph algorithms such as Kahn's Topological Sort, BFS, DFS, and Cycle Detection to analyze dependencies and generate a semester-wise course plan while considering constraints such as credit limits and course capacity.

2. Introduction

Selecting courses can become difficult when a degree program contains many prerequisite relationships. A student may need to know which course should be taken first, which courses become available after completing a particular course, whether circular prerequisites exist, and how courses can be distributed across semesters.

The Course Prerequisite Planner addresses these problems using graph theory and algorithms. Instead of manually checking every prerequisite, the application models the curriculum as a directed graph and processes the relationships algorithmically.

3. Problem Statement

Universities often have complex prerequisite structures where one course depends on another. Manually determining a valid course sequence becomes difficult as the number of courses increases. Circular prerequisites, multiple dependencies, credit limits, and course-count limits can make semester planning more complicated. Therefore, a system is required to represent, analyze, visualize, and schedule courses automatically.

4. Objectives

Represent university courses using a directed graph.

Represent prerequisite relationships between courses.

Find a valid order for completing courses.

Explore course dependencies using BFS and DFS.

Detect circular prerequisites.

Generate a semester-wise course plan.

Respect credit and course-count constraints.

Explain why a course is eligible for a semester.

Visually display the course dependency graph.

Provide algorithm execution details for learning and transparency.

5. Existing System

In a conventional approach, students generally depend on university curriculum documents, course catalogs, manual prerequisite checking, academic advisors, spreadsheets, or personal notes.

Dependencies are difficult to track when many courses are involved.

Circular dependencies can be overlooked.

Planning across multiple semesters takes time.

It is difficult to understand how one course affects later courses.

Students may need to repeatedly verify prerequisite conditions.

6. Proposed System

The proposed system uses graph theory and algorithms to automate prerequisite analysis. Each course is represented as a vertex or node, while a prerequisite relationship is represented as a directed edge. For example, $A \rightarrow B$ means that course A must be completed before course B.

The system analyzes the graph to determine valid course order, explore dependencies, detect cycles, and generate a semester-wise plan.

7. System Architecture

The overall workflow is:

Course Data $\rightarrow$ Directed Graph Creation $\rightarrow$ Graph Analysis $\rightarrow$ Cycle Detection $\rightarrow$ BFS / DFS / Topological Sort $\rightarrow$ Semester Planner $\rightarrow$ Interactive Display

8. Technologies Used

9. Data Structure Used

9.1 Directed Graph

The central data structure is a directed graph. Each course is represented as a node, and each prerequisite relationship is represented as a directed edge. For example, $A \rightarrow B$ means Course A must be completed before Course B.

10. Algorithms Used

10.1 Kahn's Topological Sort

Topological sorting finds an order in which courses can be completed while respecting prerequisite relationships. Kahn's algorithm uses in-degree values and processes courses whose in-degree is zero.

After processing a course, the in-degree of its dependent courses is reduced.

Example: Programming I $\rightarrow$ Programming II $\rightarrow$ Data Structures. A valid topological order is Programming I, Programming II, Data Structures.

10.2 Breadth-First Search (BFS)

BFS explores the graph level by level. In this project it can be used to identify downstream courses and their minimum prerequisite-hop distances.

10.3 Depth-First Search (DFS)

DFS explores one dependency path deeply before moving to another path. It is useful for exploring prerequisite chains and forms the basis of the project's cycle-detection approach.

10.4 Cycle Detection

Circular prerequisites create an impossible dependency. For example, $A \rightarrow B \rightarrow C \rightarrow A$ forms a cycle. The project uses DFS states such as unvisited, currently being explored, and completely explored to identify back-edges and circular dependencies.

11. Semester Planner

The project generates a semester-wise plan instead of only producing a course order. The planner considers course prerequisites, semester credit limits, maximum course count, course priority, and dependency relationships.

Example:

Semester 1: Programming I, Mathematics I Semester 2: Programming II, Data Structures Semester 3: Algorithms, Database Systems

12. Priority-Based Planning

When multiple courses become available at the same time, the system can use course priority. The project supports high, medium, and low priority and also considers downstream dependencies when prioritizing courses.

13. Interactive Visualization

The application provides an interactive SVG graph visualizer. It supports course nodes, directed edges, zooming, panning, dragging of nodes, dynamic edge updates, visual states, and layered graph arrangement. This helps users understand complex course dependencies visually.

14. Sample Datasets

CS Core – a standard computer-science course sequence.

Cyclic Trap – an intentionally created circular dependency for demonstrating cycle detection.

4-Year Extended – a larger example distributed across four academic tracks.

15. Project Structure

Course-Prerequisite-Planner/ `index.html` `css/` `styles.css` `js/` `models/` `Course.js` `Graph.js` `algorithms/` `topologicalSort.js` `bfs.js` `dfs.js` `cycleDetector.js` `semesterPlanner.js` `presets/` `sampleData.js` `visualizer/` `GraphRenderer.js` `app.js` `test_algorithms.js` `README.md`

16. Testing

The project includes an automated test file named `test_algorithms.js`. The repository documentation states that the test suite contains 37 tests.

Graph construction

Forward and reverse adjacency

In-degree and out-degree

Topological ordering

BFS traversal

DFS traversal

Cycle detection

Cycle-node identification

Semester capacity

Prerequisite chronology

Cyclic-plan rejection

The documented test command is: `node test_algorithms.js`

17. Advantages

Automates prerequisite planning.

Detects circular prerequisite relationships.

Provides an interactive visual representation.

Considers semester constraints.

Shows algorithm execution and intermediate states.

Can be used as an educational demonstration of graph algorithms.

18. Applications

University students

Academic advisors

Computer science learners

Curriculum designers

Educational institutions

19. Limitations

The current project is primarily an algorithm-centric prototype.

Demonstration datasets are used instead of a live university course catalog.

Course information would need updating when curricula change.

Real university scheduling can involve instructors, classrooms, timetable clashes, and course availability.

Personalized planning would require student-specific academic information.

20. Future Scope

Student login and individual course-progress tracking.

Integration with a university course database.

Personalized semester planning based on completed courses.

Timetable integration to avoid schedule conflicts.

AI-based explanations and alternative study plans.

Mobile application development.

Support for multiple universities and curriculum formats.

21. Expected Output

The expected workflow is: Course Dependency Graph → Graph Analysis → Valid Course Order → Cycle Detection → Eligible Courses → Semester-wise Plan.

The final output gives students a clearer view of prerequisite relationships and a structured plan for completing courses.

22. Conclusion

The Course Prerequisite Planner demonstrates how data structures and graph algorithms can solve a practical academic planning problem. By representing courses as nodes and prerequisites as directed edges, the system can determine valid course orders, explore dependencies, detect circular prerequisites, and generate a semester-wise plan.

A key strength of the project is algorithm transparency: it can show intermediate graph states and execution details rather than displaying only a final answer. This makes the application useful both as a planning tool and as an educational demonstration of graph algorithms.

23. One-Minute Project Explanation

Our project is a Course Prerequisite Planner that uses directed graphs to represent relationships between university courses. Each course is a node and each prerequisite is a directed edge. We use Kahn's Topological Sort to find a valid course order, BFS and DFS to explore dependencies, and cycle detection to identify impossible circular prerequisites. We also developed a semester planner that considers prerequisites, credit limits, course capacity, and priority. The system provides interactive graph visualization and explains why courses are eligible. Our goal is to convert a complex prerequisite structure into a clear and automatically generated semester plan.

24. GitHub Repository

Repository: <https://github.com/anamagandlavaishali-gif/Course-Prerequisite-Planner>

Technology | Purpose

HTML5 | Structure of the web application

CSS3 | User interface and visual design

JavaScript | Algorithms and application logic

SVG | Interactive graph visualization

GitHub | Source-code management and hosting